

Error Handling in an MCP Server with FastMCP SDK

3.4.5

Executive summary

FastMCP 3.4.5 is a **Python FastMCP release**, published on July 27, 2026. Its most error-relevant release-specific change is a `JWTVerifier` fix that skips unsupported keys in a JSON Web Key Set instead of allowing one unsupported key type, such as Ed25519, to make every token fail verification. The release also fixes Azure scope fallback and several schema-generation issues. Version 3.4.5 should therefore be pinned explicitly when reproducing the behavior described in this report. As of August 5, 2026, a newer 3.4.6 package exists, but it is outside this report's requested scope. ¹

The most important implementation rule is to preserve the distinction among three error planes:

Error plane	Meaning	Correct external representation
JSON-RPC or MCP protocol error	The request cannot be understood or dispatched as a valid MCP operation	JSON-RPC <code>error</code> object with an appropriate negative error code
Tool execution error	The MCP request is valid, but the requested operation could not complete	Successful JSON-RPC response whose <code>result.isError</code> is <code>true</code>
OAuth or HTTP authentication error	The HTTP request has no acceptable bearer token, has an invalid token, or lacks scope	HTTP <code>400</code> , <code>401</code> , or <code>403</code> , normally with OAuth fields and <code>WWW-Authenticate</code>

The MCP specification explicitly separates protocol errors from tool execution errors. Unknown tools, invalid arguments, and server-level protocol failures can use JSON-RPC errors; API failures, business failures, and model-actionable tool failures belong in a tool result with `isError: true`. This distinction matters because a language-model client can inspect a tool result, modify its arguments, wait, or select another tool, whereas a protocol error generally indicates that the MCP invocation itself was malformed or unavailable. ²

For FastMCP 3.4.5, the principal server-side APIs are:

API	Recommended use
<code>FastMCP(mask_error_details=True)</code>	Hide details of unexpected exceptions in production
<code>ToolError</code>	Raise an expected, explicitly user-safe tool failure

API	Recommended use
<code>ToolResult(..., is_error=True)</code>	Return a structured, model-actionable failure without raising
<code>ValidationError</code>	FastMCP's normalized representation of invalid tool arguments
<code>AuthorizationError</code>	Authorization failure inside FastMCP server processing
<code>McpError(ErrorData(...))</code>	Low-level MCP or JSON-RPC protocol failure
<code>@mcp.tool(timeout=...)</code>	Bound tool execution time
<code>Middleware.on_call_tool(...)</code>	Centralized logging, authorization, concurrency controls, circuit breakers, and error normalization
<code>Client.call_tool(..., raise_on_error=False)</code>	Test or consume tool failures without automatically raising
<code>CallToolResult.is_error</code>	Detect a returned tool execution failure

FastMCP's exception hierarchy includes `FastMCPError`, `ValidationError`, `ResourceError`, `ToolError`, `PromptError`, and `AuthorizationError`; `FastMCPError` carries a `log_level`, allowing expected validation failures to be logged less severely than internal faults. ³

A production server should apply the following policy:

1. Validate and normalize arguments before performing side effects.
2. Retry only failures that are reasonably transient and only within a bounded deadline.
3. Never retry a side-effecting operation unless it is naturally idempotent or protected by a durable idempotency key.
4. Apply exponential backoff with jitter, honor `Retry-After`, and place a circuit breaker around each unstable dependency.
5. Return safe, actionable error information to the model; retain stack traces and sensitive diagnostics only in server-side telemetry.
6. Validate token signature, issuer, audience, expiry, resource, and scopes; never log access or refresh tokens.
7. Instrument every request with a correlation identifier, trace, duration, result class, retry count, and dependency outcome.
8. Test malformed JSON, invalid MCP envelopes, schema failures, tool failures, timeouts, expired and revoked tokens, unsupported JWKS keys, rate limits, dependency outages, and concurrent duplicate requests.

The Node.js examples below use the official MCP TypeScript SDK rather than claiming that a Node FastMCP package has version 3.4.5. The Python package and the independently versioned TypeScript implementations are not version-equivalent. A Node implementation can nevertheless produce exactly the same MCP wire behavior and interoperate with a FastMCP 3.4.5 server. The Model Context Protocol organization maintains the official TypeScript SDK for this purpose. ⁴

Version scope and error taxonomy

Version-specific behavior

FastMCP 3.4.5 is a maintenance release in the 3.x line. Its five listed fixes are:

FastMCP 3.4.5 change	Error-handling relevance
Skip unsupported JWKS keys	Prevents a supported signing key from being ignored merely because the same JWKS contains an unsupported key
Fix Azure scope fallback	Reduces incorrect OAuth scope negotiation and authorization failures
Serialize OpenAPI <code>deepObject</code> query parameters	Reduces malformed generated upstream requests
Deterministic required-field ordering for transformed tools	Improves repeatability of generated schemas and tests
Do not mutate the caller's schema during compression	Prevents hard-to-reproduce schema and validation behavior

These changes are recorded in the official FastMCP changelog. ⁵

The immediately preceding releases also affect deployment security. FastMCP 3.4.3 hardened SSRF handling, OAuth redirect validation, and Host/Origin checks; 3.4.4 relaxed default Host/Origin behavior to restore compatibility with existing proxies and serverless deployments while leaving explicit protections available. Operators should therefore configure trusted proxy headers, public base URLs, hosts, and origins deliberately rather than assuming the framework can infer a secure reverse-proxy topology. ⁵

Error objects and their fields

JSON-RPC defines an error object with the following fields:

Field	Required	Meaning
<code>code</code>	Yes	Integer categorizing the protocol failure
<code>message</code>	Yes	Concise single-sentence description
<code>data</code>	No	Structured or primitive implementation-specific information

The standard codes are `-32700` for parse errors, `-32600` for invalid requests, `-32601` for unknown methods, `-32602` for invalid parameters, and `-32603` for internal JSON-RPC errors. The range `-32000` through `-32099` is reserved for implementation-defined server errors. A response must contain either `result` or `error`, never both. ⁶

FastMCP and MCP tool results use a different shape:

Python field in <code>ToolResult</code>	MCP wire field	Purpose
<code>content</code>	<code>content</code>	Human- and model-readable content blocks
<code>structured_content</code>	<code>structuredContent</code>	Machine-readable JSON object
<code>meta</code>	<code>_meta</code>	Runtime metadata not intended as primary tool output
<code>is_error</code>	<code>isError</code>	Signals that execution failed despite a valid MCP call

In FastMCP 3.4.5, `ToolResult.to_mcp_result()` maps these fields into an MCP `CallToolResult`. Setting `is_error=True` returns the failure as a result rather than raising it as a protocol exception. ⁷

Recommended taxonomy

Category	Examples	Detection	Retryable by server?	Recommended propagation
JSON parsing	Truncated body, illegal JSON token	Transport/JSON decoder	No	JSON-RPC <code>-32700</code> ; usually HTTP 400
Invalid JSON-RPC envelope	Missing <code>jsonrpc</code> , non-string <code>method</code> , invalid <code>id</code>	MCP transport/schema parser	No	<code>-32600</code> ; usually HTTP 400
Unsupported MCP method	Unknown method name	MCP dispatcher	No	<code>-32601</code>
Invalid tool parameters	Missing required value, wrong type, schema violation	FastMCP/Pydantic validation	No	Normally <code>-32602</code> ; where model correction is desired, a tool error result may also be appropriate
Expected tool/business error	Record absent, invalid state transition, policy rejection	Application branch or known exception	Usually no	<code>ToolError</code> or <code>ToolResult(is_error=True)</code>
Transient upstream failure	Timeout, connection reset, 429, 502, 503, 504	HTTP client or dependency adapter	Yes, bounded	Retry internally; then <code>isError: true</code> with safe recovery guidance
Permanent upstream failure	Upstream 400, unsupported operation	Dependency adapter	No	<code>isError: true</code>
Unexpected implementation fault	Null dereference, invariant failure, database bug	Uncaught exception	Usually no	Log stack; re-raise; FastMCP masks details

Category	Examples	Detection	Retryable by server?	Recommended propagation
Tool timeout	<code>@mcp.tool(timeout=...)</code> deadline exceeded	AnyIO timeout in FastMCP	Sometimes at a higher layer	MCP server error; do not blindly retry side effects
Missing or invalid bearer token	No token, bad signature, expired token, wrong audience	Auth provider or <code>JWTVerifier</code>	Refresh once if expired	HTTP 401 plus <code>WWW-Authenticate</code>
Insufficient scope	Valid token lacks permission	Scope check	Only after step-up authorization	HTTP 403 plus scope challenge
Invalid refresh token	Expired, revoked, reused, wrong client	Token endpoint	No repeated retry	OAuth <code>invalid_grant</code> ; require fresh authorization
Circuit open	Dependency failure threshold exceeded	Circuit-breaker state	No call until probe window	<code>isError: true</code> , <code>retryable: true</code> , bounded retry-after

Tool error versus protocol error

A practical classification question is: **Could the model take a sensible next action?**

If the answer is yes—change an order identifier, wait for a rate limit, ask for authorization, select another account, or obtain confirmation—return a tool execution error with concise content and a stable structured error code. If the operation could not be dispatched as MCP at all—invalid JSON, invalid envelope, unsupported method, or malformed core parameters—use a JSON-RPC error. This follows the MCP specification’s two-mechanism error model. ²

For example, these two failures should not have the same representation:

```
{
  "jsonrpc": "2.0",
  "id": 17,
  "error": {
    "code": -32602,
    "message": "Invalid params",
    "data": {
      "field": "arguments.order_id",
      "reason": "required"
    }
  }
}
```

```

{
  "jsonrpc": "2.0",
  "id": 18,
  "result": {
    "content": [
      {
        "type": "text",
        "text":
"The order service is temporarily unavailable. Retry later using the same
idempotency key."
      }
    ],
    "structuredContent": {
      "ok": false,
      "error": {
        "code": "DEPENDENCY_UNAVAILABLE",
        "retryable": true,
        "retryAfterMs": 2000
      }
    },
    "isError": true
  }
}

```

The first says the MCP invocation is invalid. The second says the invocation was valid and its execution produced a recoverable operational failure.

FastMCP APIs and propagation semantics

Exception hierarchy

FastMCP 3.4.5 defines a framework exception family rooted at `FastMCPError`. Important classes include:

Class	Typical origin	Recommended handling
<code>FastMCPError</code>	Base FastMCP failure	Catch only in framework boundaries; avoid broad application use
<code>ValidationError</code>	Invalid component or tool input	Log as client-input warning; do not retry
<code>ResourceError</code>	Resource retrieval or conversion	Convert to resource-level or protocol failure
<code>ToolError</code>	Explicit safe tool failure	Send its message to the client

Class	Typical origin	Recommended handling
<code>PromptError</code>	Prompt retrieval/ rendering	Propagate as an MCP prompt failure
<code>AuthorizationError</code>	Access-control decision	Convert to 401 or 403 depending on authentication state
<code>ClientError</code>	FastMCP client operation	Handle in tests, proxies, or downstream MCP clients
<code>NotFoundError</code>	Missing FastMCP component	Usually map to invalid params or not-found semantics
<code>DisabledError</code>	Disabled component	Return a safe unavailable/disabled response
<code>McpError</code>	Low-level MCP error	Preserve its <code>ErrorData.code</code> , message, and optional data

`FastMCPError` contains a `log_level` property. FastMCP's argument-validation path constructs its framework `ValidationError` with warning-level logging, helping separate bad client input from server defects. ⁸

Masking and explicit safe errors

FastMCP tools may raise standard Python exceptions or `ToolError`. FastMCP logs exceptions and converts them into MCP error responses. With `mask_error_details=True`, unexpected exception details are replaced with a generic client-facing message, while messages supplied through `ToolError` remain visible. Therefore, a `ToolError` message must be treated as public output and must not contain SQL text, file paths, secrets, internal hostnames, token claims, or stack-trace fragments. ⁹

Recommended initialization:

```
mcp = FastMCP(
    name="Production MCP",
    mask_error_details=True,
    strict_input_validation=True,
)
```

`mask_error_details=False` is useful only in isolated development or controlled tests. Production observability should come from private logs and traces, not from returning raw exception messages.

Structured tool errors

Use `ToolResult` when the caller benefits from a stable error contract:

```

from fastmcp.tools import ToolResult

return ToolResult(
    content=(
        "The inventory service is temporarily unavailable. "
        "Retry later with the same idempotency key."
    ),
    structured_content={
        "ok": False,
        "error": {
            "code": "INVENTORY_UNAVAILABLE",
            "category": "dependency",
            "retryable": True,
            "retryAfterMs": 2000,
        },
    },
    meta={
        "trace_id": trace_id,
        "attempts": attempts,
    },
    is_error=True,
)

```

FastMCP documents `content`, `structured_content`, and `meta` as the explicit-response controls exposed by `ToolResult`; version 3.4.5 additionally has the `is_error` field that maps to MCP `isError`. ¹⁰

A useful application-level envelope is:

```

{
  "ok": false,
  "error": {
    "code": "STABLE_MACHINE_CODE",
    "category": "validation|business|dependency|authorization|internal",
    "message": "Safe concise explanation",
    "retryable": false,
    "retryAfterMs": null,
    "fieldErrors": [],
    "remediation": "Correct the input and retry"
  }
}

```

Keep implementation diagnostics out of this envelope. Put a correlation identifier in `_meta`, but do not put secrets or raw credentials there because clients may persist or display metadata.

Validation and timeouts

FastMCP's function-tool implementation validates incoming arguments with Pydantic `TypeAdapter`. Validation failures during argument conversion become FastMCP `ValidationError` instances. By contrast, a Pydantic validation failure raised *inside* the tool body is preserved as an implementation exception, because it may indicate a server-side bug rather than bad client input. FastMCP's tool timeout uses AnyIO cancellation and constructs an `McpError` with server code `-32000` when execution exceeds the configured limit. ¹¹

This distinction should influence logging:

```
argument_validation_failure -> warning, no stack required
business_rule_rejection     -> info or warning
dependency_timeout          -> warning or error, dependency span attached
tool_body_validation_bug    -> error, stack trace and incident correlation
unexpected_exception        -> error or critical, stack trace
```

A timeout does not prove that no side effect occurred. The remote dependency might have committed an operation and lost the response. Therefore, retrying a timed-out write requires an idempotency key or reconciliation lookup.

Middleware

FastMCP middleware can override `on_call_tool(context, call_next)` and wrap tool execution. The documented way to deny a tool call from middleware is to raise `ToolError`, allowing FastMCP's normal propagation machinery to construct the client response. Returning an arbitrary `ToolResult` from access-control middleware is not recommended for denial because it can bypass expected error behavior. ¹²

Middleware is an appropriate location for:

Concern	Middleware action
Correlation	Create or extract request and trace IDs
Authorization	Check tool tags, tenant, subject, and scopes
Concurrency	Acquire per-tool or per-tenant semaphore
Rate limiting	Consume a quota token before execution
Circuit breaker	Reject calls while a dependency circuit is open
Timing	Measure queue, execution, and total duration
Normalization	Map known dependency exceptions to stable tool errors
Audit	Record subject, tool name, decision, and outcome
Redaction	Remove sensitive values from log fields

Do not place all dependency retries in a global middleware. Retry policy should normally be dependency-specific because idempotency, status-code semantics, and safe deadlines differ by operation.

Client-side detection

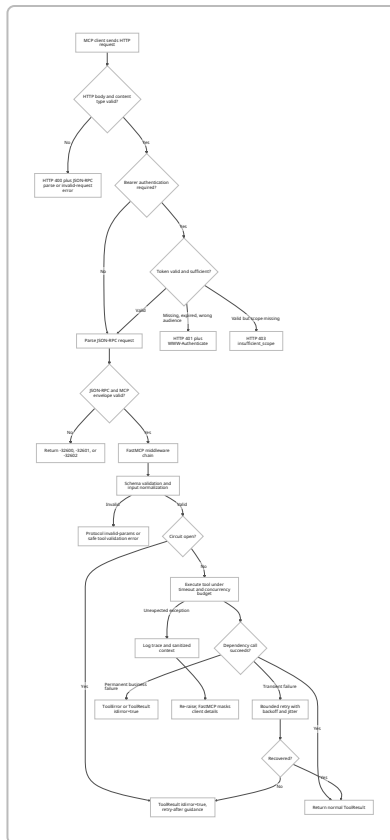
FastMCP's client defaults to raising `ToolError` when a tool result is an error. Tests and orchestrators can pass `raise_on_error=False`, inspect `result.is_error`, and then read `content`, `structured_content`, or `data`. The lower-level MCP result also exposes `is_error`. ¹³

```
async with client:
    result = await client.call_tool(
        "create_order",
        {
            "sku": "SKU-123",
            "quantity": 2,
            "idempotency_key": "4b7ddc7e-...",
        },
        raise_on_error=False,
    )

    if result.is_error:
        # Assert the machine code and safe message in tests.
        print(result.structured_content)
```

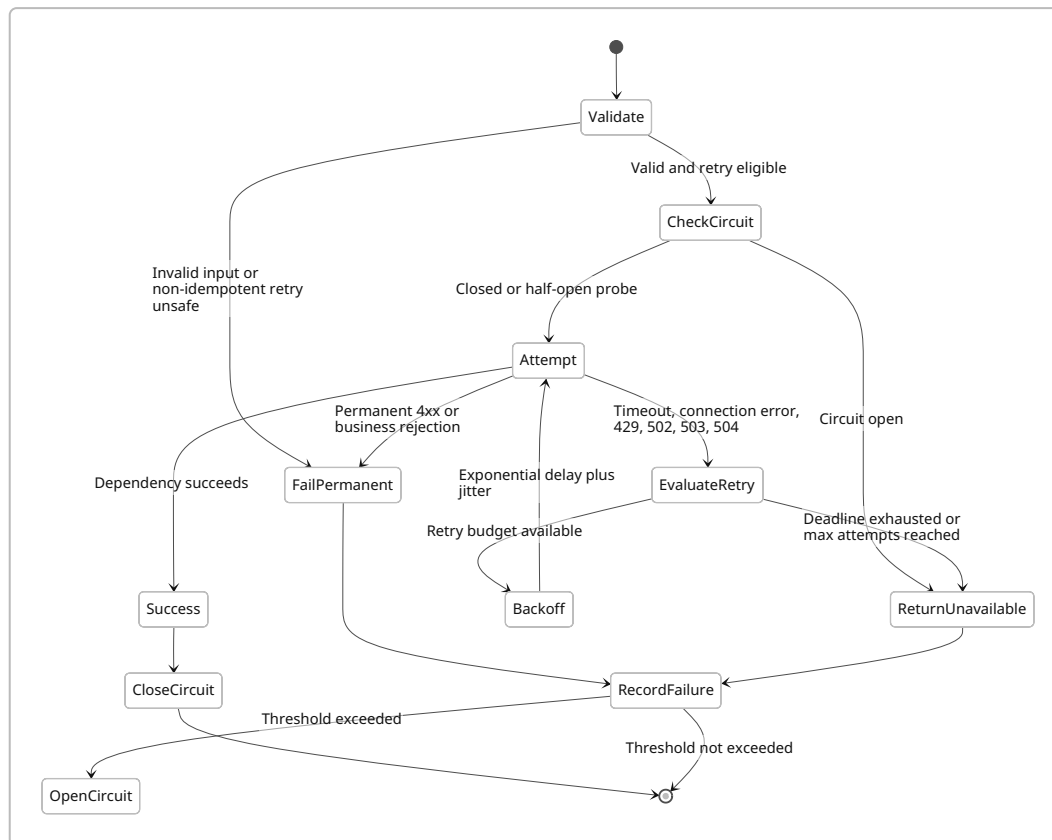
Production error-handling architecture

End-to-end error flow



The authentication decision is intentionally before JSON-RPC dispatch for HTTP transports. An unauthenticated request should receive an OAuth resource-server challenge, not a JSON-RPC tool result, because the client must first discover or complete authorization. MCP's authorization specification requires clients to process HTTP 401 responses and `WWW-Authenticate` metadata. 14

Retry and circuit-breaker logic



Retries should have four independent bounds:

Bound	Purpose
Maximum attempts	Prevent unbounded loops
Overall deadline	Prevent retries from exceeding the caller's useful latency
Per-attempt timeout	Stop a single attempt from consuming the entire deadline
Retry budget	Prevent retries from becoming a large fraction of total dependency traffic

Use exponential backoff with jitter rather than synchronized fixed delays. A retry can be safe for a read but dangerous for a write whose first response was lost. AWS's guidance emphasizes that retries rely on idempotent semantics and recommends a caller-provided unique request identifier for operations whose side effects must occur at most once. ¹⁵

Retry classification

Failure	Default policy	Rationale
DNS or connection reset	Retry if deadline permits	Commonly transient
Connect timeout	Retry	Request probably not delivered, though infrastructure-specific
Read timeout on GET/read	Retry	Read is normally idempotent
Read timeout on write	Retry only with idempotency protection	Commit outcome is unknown
HTTP 408	Retry cautiously	Server timed out request
HTTP 409	Usually no retry	Often business conflict; may retry only for documented transient lock conflict
HTTP 425	Retry after appropriate delay	Upstream indicates request is too early
HTTP 429	Honor <code>Retry-After</code> , then retry within budget	Explicit throttling
HTTP 500	Limited retry	May be transient, but repeated 500s should open circuit
HTTP 502/503/504	Retry with jitter	Gateway or availability failure
HTTP 400/404/422	No retry	Input or permanent lookup failure
HTTP 401	Refresh once when token is expired, then retry once	Repeated retry cannot repair invalid credentials
HTTP 403	No retry until scope or policy changes	Token is authenticated but unauthorized
OAuth <code>invalid_grant</code>	No retry	Refresh token or authorization grant is unusable
JSON-RPC <code>-32602</code>	No identical retry	Arguments must change
Tool <code>retryable=false</code>	No retry	Server explicitly classifies it as permanent
Circuit open	Do not call dependency	Protect dependency and server resources

Circuit-breaker design

Maintain circuit state per meaningful failure domain: typically dependency plus region, and sometimes tenant or credential pool. A single global circuit can cause one tenant's failures to deny service to every tenant. Conversely, a circuit per request has no protective value.

A practical state model is:

State	Behavior
Closed	Calls flow normally; count failures in a rolling window
Open	Calls fail fast for a configured cool-down period
Half-open	Admit a small number of probes
Closed after probe	Reset or decay failure history
Re-opened after probe	Extend cool-down, preferably with a cap

Count only dependency-health failures toward the circuit. Validation errors, business rejections, expected 404s, and insufficient-scope responses should not open it.

Idempotency

For side-effecting tools, require an idempotency key in the tool schema:

```
{
  "type": "object",
  "properties": {
    "idempotency_key": {
      "type": "string",
      "format": "uuid",
      "description": "Stable for all retries of the same logical operation"
    }
  },
  "required": ["idempotency_key"]
}
```

Store, atomically:

```
tenant_id
tool_name
idempotency_key
canonical_request_hash
state: in_progress | succeeded | failed_final
response_or_resource_reference
created_at
expires_at
```

The same key with the same canonical request should return a semantically equivalent result. The same key with materially different arguments should fail validation. Atomicity between recording the key and

committing the side effect is essential; otherwise, the server may record success without executing or execute without recording. ¹⁶

Input validation and sanitization

Validation should be semantic, not merely syntactic. FastMCP and Pydantic can enforce types and JSON Schema constraints, but application code must still validate ownership, allowed state transitions, maximum fan-out, URL destinations, file roots, query complexity, and authorization relationships. The MCP tool specification requires servers to validate inputs, enforce access controls, rate-limit invocations, and sanitize outputs. ²

Recommended controls include:

Input class	Controls
Identifiers	Length limits, explicit character set, canonical case
Free text	Unicode normalization, size limits, control-character rejection
Paths	Resolve against an allow-listed root; reject traversal and ambiguous encodings
URLs	Permit expected schemes and destinations; resolve and reject private/reserved targets where appropriate
SQL/search	Parameterized queries; query-cost and result limits
Shell/process	Avoid command construction; use argument arrays and fixed executables
File uploads	Size, type, content, decompression-ratio, and storage-path checks
Pagination	Maximum page size and cursor validation
Dates	Time-zone and range validation
Monetary values	Decimal types, currency allow-list, nonnegative and upper bounds
Tool outputs	Remove secrets, internal URLs, headers, stack traces, and untrusted active content

OWASP recommends validating input as early as possible and favors allow-list validation over attempting to enumerate every malicious form. ¹⁷

Reference implementations

Python FastMCP 3.4.5

Install and pin the requested release:

```
python -m pip install "fastmcp==3.4.5" httpx
```

The following example demonstrates validation, OAuth JWT verification, safe error responses, structured logging, bounded retries, jitter, a circuit breaker, idempotency propagation, timeout handling, and unexpected-exception masking.

```
from __future__ import annotations

import asyncio
import logging
import os
import random
import re
import time
import uuid
from dataclasses import dataclass
from typing import Annotated, Any

import httpx
from pydantic import Field

from fastmcp import Context, FastMCP
from fastmcp.exceptions import ToolError
from fastmcp.server.auth.providers.jwt import JWTVerifier
from fastmcp.tools import ToolResult
from fastmcp.utilities.logging import get_logger

logger = get_logger(__name__)

# FastMCP 3.4.5 fixed handling of unsupported keys in a JWKS.
verifier = JWTVerifier(
    jwks_uri=os.environ["JWT_JWKS_URI"],
    issuer=os.environ["JWT_ISSUER"],
    audience=os.environ["JWT_AUDIENCE"],
    required_scopes=["orders:read"],
)

mcp = FastMCP(
    name="Orders MCP",
    auth=verifier,
    mask_error_details=True,
    strict_input_validation=True,
)

ORDER_ID_RE = re.compile(r"^[A-Z0-9][A-Z0-9_-]{2,63}$")

class DependencyUnavailable(Exception):
```



```

        """Dependency failure that remained transient after bounded retries."""

class CircuitOpen(DependencyUnavailable):
    pass

@dataclass
class CircuitBreaker:
    failure_threshold: int = 5
    reset_after_seconds: float = 20.0

    failures: int = 0
    opened_at: float | None = None
    half_open_probe_in_flight: bool = False

    def before_call(self) -> None:
        if self.opened_at is None:
            return

        elapsed = time.monotonic() - self.opened_at
        if elapsed < self.reset_after_seconds:
            raise CircuitOpen("orders dependency circuit is open")

        # Half-open: permit one probe.
        if self.half_open_probe_in_flight:
            raise CircuitOpen("orders dependency circuit is half-open")

        self.half_open_probe_in_flight = True

    def success(self) -> None:
        self.failures = 0
        self.opened_at = None
        self.half_open_probe_in_flight = False

    def transient_failure(self) -> None:
        self.half_open_probe_in_flight = False
        self.failures += 1
        if self.failures >= self.failure_threshold:
            self.opened_at = time.monotonic()

breaker = CircuitBreaker()
http_client = httpx.AsyncClient(
    base_url=os.environ.get("ORDERS_API_URL", "https://
orders.internal.example"),
    timeout=httpx.Timeout(connect=2.0, read=4.0, write=4.0, pool=1.0),
    limits=httpx.Limits(max_connections=100, max_keepalive_connections=20),

```

```

)

def safe_order_id(raw: str) -> str:
    """Canonicalize, then validate; never silently remove meaningful input."""
    value = raw.strip().upper()

    if any(ord(ch) < 32 for ch in value):
        raise ValueError("control characters are not allowed")

    if not ORDER_ID_RE.fullmatch(value):
        raise ValueError(
            "order_id must be 3-64 characters using A-Z, 0-9, '_' or '-'")
    )

    return value

def parse_retry_after_seconds(response: httpx.Response) -> float | None:
    value = response.headers.get("retry-after")
    if value is None:
        return None

    # This sample supports delta-seconds. A production implementation can
    # additionally parse an HTTP-date.
    try:
        return max(0.0, min(float(value), 30.0))
    except ValueError:
        return None

def is_transient_status(status: int) -> bool:
    return status in {408, 425, 429, 500, 502, 503, 504}

async def get_order_from_upstream(
    *,
    order_id: str,
    idempotency_key: str,
    trace_id: str,
    max_attempts: int = 3,
    deadline_seconds: float = 8.0,
) -> tuple[dict[str, Any], int]:
    """
    Retry transient read failures using bounded exponential backoff with jitter.
    The idempotency key is forwarded even though this example is a read, so
    the same helper pattern can safely support idempotent writes.
    """

```

```

breaker.before_call()
deadline = time.monotonic() + deadline_seconds
last_error: Exception | None = None

for attempt in range(1, max_attempts + 1):
    remaining = deadline - time.monotonic()
    if remaining <= 0:
        break

    try:
        response = await http_client.get(
            f"/v1/orders/{order_id}",
            headers={
                "X-Request-ID": trace_id,
                "Idempotency-Key": idempotency_key,
            },
            timeout=min(remaining, 4.0),
        )

        if response.status_code == 404:
            # Permanent business result. Do not count toward the circuit.
            raise ToolError(f"Order {order_id} was not found.")

        if response.status_code == 403:
            raise ToolError(
                "You are authenticated but are not permitted to view this
order."
            )

        if is_transient_status(response.status_code):
            retry_after = parse_retry_after_seconds(response)
            raise DependencyUnavailable(
                f"transient upstream status={response.status_code};"
                f" retry_after={retry_after}"
            )

        response.raise_for_status()
        payload = response.json()

        # Validate important response invariants before exposing output.
        if not isinstance(payload, dict) or payload.get("id") != order_id:
            raise RuntimeError("orders API returned an invalid response
shape")

        breaker.success()
        return payload, attempt

    except ToolError:

```

```

        # Expected permanent result: do not wrap or retry.
        raise

    except (
        httpx.ConnectError,
        httpx.ConnectTimeout,
        httpx.ReadTimeout,
        DependencyUnavailable,
    ) as exc:
        last_error = exc
        breaker.transient_failure()

        if attempt >= max_attempts:
            break

        remaining = deadline - time.monotonic()
        if remaining <= 0:
            break

        # Capped exponential backoff with full jitter.
        cap = min(0.25 * (2 ** (attempt - 1)), 2.0)
        delay = random.uniform(0.0, cap)

        # Prefer Retry-After where the dependency supplied it.
        if isinstance(exc, DependencyUnavailable):
            match = re.search(r"retry_after=([0-9.]+)", str(exc))
            if match:
                delay = max(delay, min(float(match.group(1)), 30.0))

        await asyncio.sleep(min(delay, max(0.0, remaining)))

    raise DependencyUnavailable(
        f"orders dependency failed after {max_attempts} attempts"
    ) from last_error

def dependency_error_result(
    *,
    trace_id: str,
    attempts: int,
    retry_after_ms: int,
) -> ToolResult:
    return ToolResult(
        content=(
            "The order service is temporarily unavailable. "
            "Retry later using the same idempotency key."
        ),
        structured_content={

```

```

        "ok": False,
        "error": {
            "code": "ORDERS_DEPENDENCY_UNAVAILABLE",
            "category": "dependency",
            "message": "The order service is temporarily unavailable.",
            "retryable": True,
            "retryAfterMs": retry_after_ms,
            "remediation": (
                "Wait for the indicated interval and retry with the same "
                "idempotency key."
            ),
        },
    },
    meta={
        "trace_id": trace_id,
        "attempts": attempts,
    },
    is_error=True,
)

@mcp.tool(
    timeout=12.0,
    output_schema=None,
)
async def get_order(
    order_id: Annotated[str, Field(min_length=3, max_length=64)],
    idempotency_key: Annotated[str, Field(min_length=16, max_length=128)],
    ctx: Context,
) -> ToolResult:
    """
    Retrieve an order.

    idempotency_key must remain unchanged across retries of the same logical
    call.
    """
    trace_id = uuid.uuid4().hex
    started = time.monotonic()
    attempts = 0

    try:
        normalized_id = safe_order_id(order_id)

        # Validate UUID format without returning its raw parse error.
        try:
            uuid.UUID(idempotency_key)
        except ValueError as exc:
            raise ToolError(

```

```

        "idempotency_key must be a valid UUID and must be reused on
retries."
    ) from exc

    await ctx.info(
        "Starting order lookup",
        extra={
            "trace_id": trace_id,
            "tool": "get_order",
            "order_id": normalized_id,
            # Do not log bearer tokens or full sensitive tool arguments.
        },
    )

    payload, attempts = await get_order_from_upstream(
        order_id=normalized_id,
        idempotency_key=idempotency_key,
        trace_id=trace_id,
    )

    elapsed_ms = round((time.monotonic() - started) * 1000)

    logger.info(
        "tool_completed",
        extra={
            "event": "tool_completed",
            "trace_id": trace_id,
            "tool": "get_order",
            "outcome": "success",
            "attempts": attempts,
            "latency_ms": elapsed_ms,
        },
    )

    return ToolResult(
        content=f"Order {normalized_id} was retrieved successfully.",
        structured_content={
            "ok": True,
            "order": {
                "id": payload["id"],
                "status": payload.get("status"),
                "updatedAt": payload.get("updatedAt"),
            },
        },
        meta={
            "trace_id": trace_id,
            "attempts": attempts,
            "latency_ms": elapsed_ms,

```

```

        },
    )

except ValueError as exc:
    # A deliberately safe validation message.
    raise ToolError(f"Invalid order_id: {exc}") from exc

except CircuitOpen:
    logger.warning(
        "tool_rejected_circuit_open",
        extra={
            "event": "tool_rejected",
            "trace_id": trace_id,
            "tool": "get_order",
            "error_class": "circuit_open",
        },
    )
    return dependency_error_result(
        trace_id=trace_id,
        attempts=attempts,
        retry_after_ms=20_000,
    )

except DependencyUnavailable:
    logger.warning(
        "tool_dependency_failure",
        extra={
            "event": "tool_failed",
            "trace_id": trace_id,
            "tool": "get_order",
            "error_class": "dependency_unavailable",
            "attempts": attempts,
        },
    )
    return dependency_error_result(
        trace_id=trace_id,
        attempts=max(attempts, 3),
        retry_after_ms=2_000,
    )

except ToolError:
    # FastMCP will propagate the deliberately safe message.
    raise

except Exception:
    # Full stack remains private. mask_error_details=True protects clients.
    logger.exception(
        "unexpected_tool_failure",

```

```

        extra={
            "event": "tool_failed",
            "trace_id": trace_id,
            "tool": "get_order",
            "error_class": "internal",
        },
    )
    raise

if __name__ == "__main__":
    try:
        mcp.run(transport="http", host="0.0.0.0", port=8000)
    finally:
        # In a production ASGI deployment, close this in the application
        lifespan.
        asyncio.run(http_client.aclose())

```

The JWT verifier configuration follows FastMCP's documented `JWTVerifier(jwks_uri, issuer, audience, required_scopes)` pattern. FastMCP recommends JWKS integration for automatic key rotation and permits production settings to be loaded from environment variables. ¹⁸

The code returns expected dependency exhaustion as `ToolResult(is_error=True)`, raises safe validation and business failures as `ToolError`, and re-raises unexpected faults so that FastMCP can log them and mask their client-facing details. This mirrors FastMCP's documented server and result semantics.

¹⁹

Python error-handling middleware

For larger servers, put correlation, timing, and normalization in middleware:

```

import time
import uuid

from fastmcp.exceptions import ToolError
from fastmcp.server.middleware import Middleware, MiddlewareContext

class ErrorBoundaryMiddleware(Middleware):
    async def on_call_tool(self, context: MiddlewareContext, call_next):
        trace_id = uuid.uuid4().hex
        started = time.monotonic()
        tool_name = context.message.name

        try:
            result = await call_next(context)

```



```

        logger.info(
            "mcp_tool_result",
            extra={
                "event": "mcp_tool_result",
                "trace_id": trace_id,
                "tool": tool_name,
                "is_error": bool(getattr(result, "is_error", False)),
                "latency_ms": round(
                    (time.monotonic() - started) * 1000
                ),
            },
        )
    )
    return result

except ToolError:
    logger.warning(
        "mcp_tool_expected_error",
        extra={
            "event": "mcp_tool_error",
            "trace_id": trace_id,
            "tool": tool_name,
            "error_class": "expected",
        },
    )
    raise

except Exception:
    logger.exception(
        "mcp_tool_unexpected_error",
        extra={
            "event": "mcp_tool_error",
            "trace_id": trace_id,
            "tool": tool_name,
            "error_class": "internal",
        },
    )
    raise

```

Register this middleware through the `middleware` argument to `FastMCP` or the server's middleware registration API, depending on how the application is assembled. FastMCP's middleware contract uses `Middleware`, `MiddlewareContext`, and `call_next`; authorization denial from middleware should raise `ToolError`. ¹²

Node.js protocol-equivalent server

There is no Python FastMCP SDK 3.4.5 runtime for Node.js. The following example uses the official MCP TypeScript SDK and produces the same `isError` semantics. The exact package major should be pinned in the Node lockfile because the TypeScript SDK evolves independently of Python FastMCP. ²⁰

```
npm install @modelcontextprotocol/sdk zod jose pino
```

```
import { randomUUID } from "node:crypto";
import { setTimeout as sleep } from "node:timers/promises";

import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import {
  ErrorCode,
  McpError,
} from "@modelcontextprotocol/sdk/types.js";
import { z } from "zod";
import pino from "pino";

const log = pino({
  level: process.env.LOG_LEVEL ?? "info",
  // Production deployments should add explicit redaction paths for headers,
  // cookies, authorization fields, and known token properties.
  redact: [
    "req.headers.authorization",
    "/*.access_token",
    "/*.refresh_token",
    "/*.client_secret",
  ],
});

const server = new McpServer({
  name: "orders-node-companion",
  version: "1.0.0",
});

class CircuitOpenError extends Error {}
class TransientDependencyError extends Error {}

class CircuitBreaker {
  private failures = 0;
  private openedAtMs: number | null = null;
  private probeInFlight = false;
```

```

constructor(
  private readonly threshold = 5,
  private readonly resetAfterMs = 20_000,
) {}

beforeCall(): void {
  if (this.openedAtMs === null) return;

  if (Date.now() - this.openedAtMs < this.resetAfterMs) {
    throw new CircuitOpenError("Circuit open");
  }

  if (this.probeInFlight) {
    throw new CircuitOpenError("Half-open probe already running");
  }

  this.probeInFlight = true;
}

success(): void {
  this.failures = 0;
  this.openedAtMs = null;
  this.probeInFlight = false;
}

transientFailure(): void {
  this.probeInFlight = false;
  this.failures += 1;

  if (this.failures >= this.threshold) {
    this.openedAtMs = Date.now();
  }
}

}

const breaker = new CircuitBreaker();

function fullJitterDelayMs(attempt: number): number {
  const cap = Math.min(250 * 2 ** (attempt - 1), 2_000);
  return Math.floor(Math.random() * cap);
}

function transientStatus(status: number): boolean {
  return [408, 425, 429, 500, 502, 503, 504].includes(status);
}

```

```

async function fetchOrder(
  orderId: string,
  idempotencyKey: string,
  traceId: string,
): Promise<{ order: unknown; attempts: number }> {
  breaker.beforeCall();

  const maxAttempts = 3;
  const deadline = Date.now() + 8_000;
  let lastError: unknown;

  for (let attempt = 1; attempt <= maxAttempts; attempt += 1) {
    const remainingMs = deadline - Date.now();
    if (remainingMs <= 0) break;

    const controller = new AbortController();
    const timeout = setTimeout(
      () => controller.abort(),
      Math.min(4_000, remainingMs),
    );

    try {
      const response = await fetch(
        `${process.env.ORDERS_API_URL}/v1/orders/${encodeURIComponent(orderId)}`,
        {
          method: "GET",
          headers: {
            "X-Request-ID": traceId,
            "Idempotency-Key": idempotencyKey,
          },
          signal: controller.signal,
        },
      );

      if (response.status === 404) {
        return {
          order: { kind: "not_found" },
          attempts: attempt,
        };
      }

      if (response.status === 403) {
        return {
          order: { kind: "forbidden" },
          attempts: attempt,
        };
      }
    }
  }
}

```

```

    }

    if (transientStatus(response.status)) {
        throw new TransientDependencyError(
            `Transient status ${response.status}`,
        );
    }

    if (!response.ok) {
        // Permanent upstream error: no retry.
        throw new McpError(
            ErrorCode.InternalError,
            "The order provider rejected the request.",
        );
    }

    const payload: unknown = await response.json();
    breaker.success();
    return { order: payload, attempts: attempt };
} catch (error) {
    lastError = error;

    if (error instanceof McpError) {
        throw error;
    }

    breaker.transientFailure();

    if (attempt >= maxAttempts || Date.now() >= deadline) {
        break;
    }

    await sleep(fullJitterDelayMs(attempt));

} finally {
    clearTimeout(timeout);
}

throw new TransientDependencyError(
    `Dependency unavailable: ${String(lastError)}`,
);
}

server.registerTool(
    "get_order",

```

```

{
  title: "Get order",
  description: "Retrieve an order by ID.",
  inputSchema: {
    order_id: z
      .string()
      .trim()
      .regex(/^[A-Z0-9][A-Z0-9_-]{2,63}$/),
    idempotency_key: z.string().uuid(),
  },
},
},
async ({ order_id, idempotency_key }) => {
  const traceId = randomUUID().replaceAll("-", "");
  const started = performance.now();

  log.info(
    {
      event: "tool_started",
      trace_id: traceId,
      tool: "get_order",
      order_id,
    },
    "Starting tool execution",
  );

  try {
    const { order, attempts } = await fetchOrder(
      order_id,
      idempotency_key,
      traceId,
    );

    if (
      typeof order === "object" &&
      order !== null &&
      "kind" in order &&
      order.kind === "not_found"
    ) {
      return {
        content: [
          {
            type: "text" as const,
            text: `Order ${order_id} was not found.`,
          },
        ],
        structuredContent: {
          ok: false,
          error: {

```

```

        code: "ORDER_NOT_FOUND",
        category: "business",
        retryable: false,
    },
},
isError: true,
};
}

if (
    typeof order === "object" &&
    order !== null &&
    "kind" in order &&
    order.kind === "forbidden"
) {
    return {
        content: [
            {
                type: "text" as const,
                text: "You are not permitted to view this order.",
            },
        ],
        structuredContent: {
            ok: false,
            error: {
                code: "ORDER_FORBIDDEN",
                category: "authorization",
                retryable: false,
            },
        },
        isError: true,
    };
}

log.info(
    {
        event: "tool_completed",
        trace_id: traceId,
        tool: "get_order",
        outcome: "success",
        attempts,
        latency_ms: Math.round(performance.now() - started),
    },
    "Tool completed",
);

return {
    content: [

```

```

        {
            type: "text" as const,
            text: `Order ${order_id} was retrieved successfully.`,
        },
    ],
    structuredContent: {
        ok: true,
        order,
    },
    _meta: {
        trace_id: traceId,
        attempts,
    },
};

} catch (error) {
    if (
        error instanceof CircuitOpenError ||
        error instanceof TransientDependencyError
    ) {
        log.warn(
            {
                event: "tool_failed",
                trace_id: traceId,
                tool: "get_order",
                error_class:
                    error instanceof CircuitOpenError
                        ? "circuit_open"
                        : "dependency_unavailable",
                latency_ms: Math.round(performance.now() - started),
            },
            "Recoverable tool failure",
        );

        return {
            content: [
                {
                    type: "text" as const,
                    text:
                        "The order service is temporarily unavailable. " +
                        "Retry later with the same idempotency key.",
                },
            ],
            structuredContent: {
                ok: false,
                error: {
                    code: "ORDERS_DEPENDENCY_UNAVAILABLE",
                    category: "dependency",
                },
            },
        };
    }
}

```



```

        retryable: true,
        retryAfterMs:
            error instanceof CircuitOpenError ? 20_000 : 2_000,
    },
},
_meta: {
    trace_id: traceId,
},
isError: true,
};
}

if (error instanceof McpError) {
    log.warn(
        {
            event: "mcp_protocol_error",
            trace_id: traceId,
            code: error.code,
        },
        "MCP error",
    );
    throw error;
}

log.error(
    {
        event: "tool_failed",
        trace_id: traceId,
        tool: "get_order",
        error_class: "internal",
        err: error,
    },
    "Unexpected tool failure",
);

// Do not put error.message into a client response.
throw new McpError(
    ErrorCode.InternalError,
    "The tool could not complete because of an internal error.",
);
}
},
);

```

For invalid tool arguments, the SDK's schema layer should normally produce invalid-parameter behavior. If custom dispatch code detects a malformed protocol-level parameter, throw an `McpError` with the SDK's invalid-parameter code rather than returning a business error:

```
throw new McpError(
  ErrorCode.InvalidParams,
  "order_id is required and must match the documented schema.",
);
```

For an expected execution failure, return:

```
return {
  content: [
    {
      type: "text",
      text: "The account is locked. Ask the user to unlock it before retrying.",
    },
  ],
  structuredContent: {
    ok: false,
    error: {
      code: "ACCOUNT_LOCKED",
      retryable: false,
    },
  },
  isError: true,
};
```

The latter matches MCP's standardized tool-execution error mechanism. ²

Node OAuth verification boundary

A Node gateway in front of FastMCP can verify bearer tokens and return HTTP OAuth errors before forwarding MCP traffic:

```
import {
  createRemoteJWKSet,
  jwtVerify,
  errors as joseErrors,
} from "jose";

const issuer = new URL(process.env.JWT_ISSUER!);
const audience = process.env.JWT_AUDIENCE!;
const jwks = createRemoteJWKSet(
  new URL(process.env.JWT_JWKS_URI!),
);

type AuthResult =
```

```

    | {
      ok: true;
      subject: string;
      scopes: Set<string>;
    }
    | {
      ok: false;
      status: 401 | 403;
      oauthError: "invalid_token" | "insufficient_scope";
      description: string;
    };

export async function authenticate(
  authorizationHeader: string | undefined,
  requiredScopes: string[],
): Promise<AuthResult> {
  if (!authorizationHeader?.startsWith("Bearer ")) {
    return {
      ok: false,
      status: 401,
      oauthError: "invalid_token",
      description: "Authentication is required.",
    };
  }

  const token = authorizationHeader.slice("Bearer ".length);

  try {
    const { payload } = await jwtVerify(token, jwks, {
      issuer: issuer.href.replace(/\/$/, ""),
      audience,
    });

    const scopeValue =
      typeof payload.scope === "string" ? payload.scope : "";
    const scopes = new Set(scopeValue.split(/\s+/).filter(Boolean));
    const missing = requiredScopes.filter((scope) => !scopes.has(scope));

    if (missing.length > 0) {
      return {
        ok: false,
        status: 403,
        oauthError: "insufficient_scope",
        description: `Required scope is missing: ${missing.join(" ")}`,
      };
    }

    if (typeof payload.sub !== "string" || payload.sub.length === 0) {

```

```

        return {
            ok: false,
            status: 401,
            oauthError: "invalid_token",
            description: "The token has no acceptable subject.",
        };
    }

    return {
        ok: true,
        subject: payload.sub,
        scopes,
    };
} catch (error) {
    const reason =
        error instanceof joseErrors.JWTExpired
            ? "The access token expired."
            : "The access token is invalid.";

    // Log only error classification, issuer and trace ID—not the token.
    log.warn(
        {
            event: "oauth_validation_failed",
            reason:
                error instanceof joseErrors.JWTExpired
                    ? "expired"
                    : "invalid",
        },
        "OAuth token rejected",
    );

    return {
        ok: false,
        status: 401,
        oauthError: "invalid_token",
        description: reason,
    };
}
}

```

The HTTP adapter should use that result to emit `WWW-Authenticate` and should never redirect an MCP API request to a login page.

HTTP, JSON-RPC, and OAuth response mapping

Recommended status mapping

The following is a design mapping, not a claim that FastMCP automatically chooses every listed HTTP status. JSON-RPC is transport-agnostic, so the HTTP adapter and reverse proxy must preserve the distinction between HTTP transport failures and JSON-RPC results. ⁶

Internal condition	SDK or wire object	HTTP status	Client action
Invalid JSON body	JSON-RPC <code>-32700</code>	400	Correct serialization; do not retry identical body
Invalid JSON-RPC object	<code>-32600</code>	400	Correct request envelope
Unknown MCP method	<code>-32601</code>	Usually 200 once parsed as JSON-RPC	Upgrade or correct client method
Invalid MCP/tool params	<code>-32602</code> or FastMCP <code>ValidationError</code>	Usually 200 after valid JSON-RPC dispatch	Correct arguments
Tool <code>ToolError</code>	Tool failure converted by FastMCP	200	Inspect safe content; modify request if actionable
<code>ToolResult(is_error=True)</code>	<code>result.isError=true</code>	200	Inspect structured error and retry policy
Internal MCP error	<code>-32603</code> or implementation server error	Usually 200 if JSON-RPC layer remains healthy	Do not immediately retry unless documented
Tool timeout	FastMCP <code>McpError</code> , often server code <code>-32000</code>	Usually 200	Reconcile write outcome before retry
Malformed bearer usage	OAuth <code>invalid_request</code>	400	Correct Authorization header

Internal condition	SDK or wire object	HTTP status	Client action
Missing/expired/invalid token	OAuth <code>invalid_token</code>	401	Discover auth metadata or refresh/re-authenticate
Valid token, missing scope	OAuth <code>insufficient_scope</code>	403	Obtain challenged scope
Invalid OAuth token request	<code>invalid_request</code> , <code>invalid_grant</code> , etc.	400	Correct request or restart authorization
Invalid OAuth client authentication	<code>invalid_client</code>	401 where HTTP authentication was attempted	Correct client authentication
Rate limit before MCP acceptance	Gateway rejection	429	Honor <code>Retry-After</code>
Per-tool upstream rate limit	<code>ToolResult(is_error=True)</code>	200	Honor tool <code>retryAfterMs</code>
Server draining or unavailable	Transport failure	503	Retry another replica with jitter
Payload too large	HTTP transport rejection	413	Reduce request size
Unsupported media type	HTTP transport rejection	415	Send accepted MCP content type

A key analytical point is that **HTTP 200 does not imply tool success**. It can mean the JSON-RPC exchange was successfully transported while the tool's result has `isError: true` or the JSON-RPC response contains an `error`. Clients must parse the MCP or JSON-RPC body, not classify outcomes from HTTP status alone.

Sample JSON-RPC responses

Parse error:

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

{
```

```
"jsonrpc": "2.0",
"id": null,
"error": {
  "code": -32700,
  "message": "Parse error"
}
}
```

Invalid request:

```
HTTP/1.1 400 Bad Request
Content-Type: application/json

{
  "jsonrpc": "2.0",
  "id": null,
  "error": {
    "code": -32600,
    "message": "Invalid Request"
  }
}
```

Invalid parameters:

```
HTTP/1.1 200 OK
Content-Type: application/json

{
  "jsonrpc": "2.0",
  "id": 42,
  "error": {
    "code": -32602,
    "message": "Invalid params",
    "data": {
      "fieldErrors": [
        {
          "field": "arguments.quantity",
          "reason": "must be greater than zero"
        }
      ]
    }
  }
}
```

Tool execution failure:

```

HTTP/1.1 200 OK
Content-Type: application/json

{
  "jsonrpc": "2.0",
  "id": 43,
  "result": {
    "content": [
      {
        "type": "text",
        "text": "The inventory service is rate limited. Retry after five
seconds."
      }
    ],
    "structuredContent": {
      "ok": false,
      "error": {
        "code": "INVENTORY_RATE_LIMITED",
        "category": "dependency",
        "retryable": true,
        "retryAfterMs": 5000
      }
    },
    "_meta": {
      "trace_id": "41a4799729c24ca1a91040f31cd1558b"
    },
    "isError": true
  }
}

```

These structures follow the JSON-RPC error object and MCP tool-result formats. ²¹

OAuth resource-server errors

RFC 6750 defines bearer-token resource errors:

OAuth error	Status	Meaning
<code>invalid_request</code>	400	Malformed bearer-token request, repeated token, or multiple token mechanisms
<code>invalid_token</code>	401	Expired, revoked, malformed, or otherwise unacceptable token
<code>insufficient_scope</code>	403	Valid token does not grant the required permission

RFC 6750 specifies that an expired access token is answered with HTTP 401 and a bearer challenge containing `error="invalid_token"`. ²²

Missing or invalid token:

```
HTTP/1.1 401 Unauthorized
WWW-Authenticate: Bearer
  resource_metadata="https://mcp.example.com/.well-known/oauth-protected-
resource",
  error="invalid_token",
  error_description="Authentication is required"
Content-Type: application/json
Cache-Control: no-store

{
  "error": "invalid_token",
  "error_description": "Authentication is required"
}
```

Insufficient scope:

```
HTTP/1.1 403 Forbidden
WWW-Authenticate: Bearer
  resource_metadata="https://mcp.example.com/.well-known/oauth-protected-
resource",
  error="insufficient_scope",
  scope="orders:write"
Content-Type: application/json
Cache-Control: no-store

{
  "error": "insufficient_scope",
  "error_description": "The orders:write scope is required"
}
```

RFC 9728 defines the `resource_metadata` parameter in `WWW-Authenticate`; MCP authorization requires clients to consume this challenge and retrieve protected-resource metadata. ²³

OAuth token-endpoint errors

At the token endpoint, RFC 6749 defines:

Error	Typical cause	Default status	Retry behavior
<code>invalid_request</code>	Missing or repeated parameter, malformed request	400	Correct request

Error	Typical cause	Default status	Retry behavior
<code>invalid_client</code>	Client authentication failed	400 or 401	Correct credentials; do not loop
<code>invalid_grant</code>	Expired, revoked, reused, mismatched, or wrong-client grant/refresh token	400	Restart authorization
<code>unauthorized_client</code>	Client cannot use requested grant	400	Reconfigure client
<code>unsupported_grant_type</code>	Grant unsupported	400	Select supported grant
<code>invalid_scope</code>	Requested scope invalid or excessive	400	Request acceptable scopes

RFC 6749 states that token endpoint errors normally use HTTP 400, while `invalid_client` may use 401 and must use `WWW-Authenticate` when client authentication was attempted through the Authorization header. It defines `invalid_grant` to include invalid, expired, revoked, redirect-mismatched, or wrong-client authorization grants and refresh tokens. ²⁴

Example:

```
HTTP/1.1 400 Bad Request
Content-Type: application/json
Cache-Control: no-store
Pragma: no-cache

{
  "error": "invalid_grant",
  "error_description": "The refresh token is invalid or no longer active"
}
```

Do not reveal whether a refresh token belonged to a particular user, when it was revoked, or which exact validation branch failed.

FastMCP OAuth APIs

FastMCP exposes several authentication models:

Class	Use case
<code>TokenVerifier</code>	Abstract validation of tokens issued elsewhere
<code>JWTVerifier</code>	Validate JWT signatures and claims through JWKS or configured keys

Class	Use case
<code>RemoteAuthProvider</code>	Use an external provider that supports Dynamic Client Registration
<code>OAuthProxy</code>	Bridge an upstream provider that lacks MCP-compatible DCR
<code>OAuthProvider</code>	Implement a fuller OAuth authorization-server role
<code>MultiAuth</code>	Accept multiple verification or auth sources

FastMCP documentation recommends `RemoteAuthProvider` for identity providers with DCR and `OAuthProxy` for traditional providers without DCR. Authentication applies to HTTP-based transports; stdio inherits the security context of the local process. ²⁵

A FastMCP OAuth proxy is configured along these lines:

```
from fastmcp import FastMCP
from fastmcp.server.auth import OAuthProxy
from fastmcp.server.auth.providers.jwt import JWTVerifier

token_verifier = JWTVerifier(
    jwks_uri="https://idp.example.com/.well-known/jwks.json",
    issuer="https://idp.example.com",
    audience="orders-mcp",
)

auth = OAuthProxy(
    upstream_authorization_endpoint="https://idp.example.com/oauth/authorize",
    upstream_token_endpoint="https://idp.example.com/oauth/token",
    upstream_client_id=os.environ["OAUTH_CLIENT_ID"],
    upstream_client_secret=os.environ["OAUTH_CLIENT_SECRET"],
    token_verifier=token_verifier,
    base_url="https://mcp.example.com",
)

mcp = FastMCP(
    name="Protected Orders MCP",
    auth=auth,
    mask_error_details=True,
)
```

This constructor pattern is documented by FastMCP. ²⁶

Token handling, refresh, and revocation

FastMCP's OAuth proxy uses a token-factory design: it retains encrypted upstream credentials and issues server-specific FastMCP tokens rather than exposing an upstream provider token directly to the MCP client.

The documentation describes audience validation, JTI-based lookup, upstream token refresh, and alignment between the client-facing and upstream token lifecycle. ²⁶

Production token policy should include:

Control	Requirement
Transmission	HTTPS only; never permit bearer credentials over plaintext
Storage	Encrypted at rest; restrict key and database access
Logging	Redact Authorization, cookies, access tokens, refresh tokens, codes, client secrets, and assertions
JWT validation	Restrict algorithms; validate signature, <code>iss</code> , <code>aud</code> , expiry, not-before, resource, and scope
JWKS	Cache with bounded lifetime, support rotation, reject ambiguous key selection
Refresh	Single-flight refresh per token/grant to prevent concurrent refresh races
Replay	Use refresh-token rotation or sender-constrained tokens where supported
Revocation	Revoke the grant family when refresh-token reuse indicates compromise
Audience	Issue and accept tokens only for the intended MCP resource
Scope	Require the minimum scopes and support incremental authorization
Redirects	Exact registered matching; reject unsafe schemes and untrusted wildcard domains
Error details	Return broad OAuth categories; keep internal reason codes private

OAuth Security Best Current Practice requires public-client refresh tokens to use replay detection through sender-constraining or rotation, and recommends audience-restricted and, where feasible, sender-constrained access tokens. ²⁷

RFC 7009 deliberately returns HTTP 200 whether a submitted token was successfully revoked or was already invalid, reducing token-state probing. The defined revocation-specific error is `unsupported_token_type`. ²⁸

Example revocation response:

```
HTTP/1.1 200 OK
Cache-Control: no-store
```

Do not return `404 token not found` from a standards-compatible revocation endpoint.

Observability, testing, and runtime operations

Structured logs

FastMCP's `Context` methods—`debug`, `info`, `warning`, `error`, and `log`—accept an `extra` dictionary for structured client-directed logging. Messages sent to clients are also written to server logs at debug level. Private operational diagnostics should use normal server-side logging rather than client-directed messages. ²⁹

A recommended log schema is:

```
{
  "timestamp": "2026-08-05T15:04:17.395Z",
  "severity": "WARN",
  "event": "mcp_tool_failed",
  "service": "orders-mcp",
  "service_version": "3.4.5-app.12",
  "environment": "production",
  "trace_id": "41a4799729c24ca1a91040f31cd1558b",
  "span_id": "3fd5f770d4ee2ab1",
  "request_id": "mcp-017f3c",
  "jsonrpc_id_type": "string",
  "mcp_method": "tools/call",
  "tool": "get_order",
  "tenant_hash": "tnt_34b6d1",
  "outcome": "error",
  "error_plane": "tool",
  "error_code": "ORDERS_DEPENDENCY_UNAVAILABLE",
  "error_class": "dependency",
  "retryable": true,
  "attempt": 3,
  "circuit_state": "closed",
  "latency_ms": 7842,
  "queue_ms": 4,
  "dependency": "orders-api",
  "dependency_status": 503
}
```

Do not log:

```
{
  "authorization": "Bearer eyJ...",
  "access_token": "...",
  "refresh_token": "...",
  "authorization_code": "...",
}
```

```

    "client_secret": "...",
    "cookie": "...",
    "tool_arguments": {
      "password": "...",
      "private_key": "...",
      "document_contents": "..."
    }
  }
}

```

OWASP recommends excluding access tokens, authentication passwords, encryption keys, connection strings, and other primary secrets from application logs. ³⁰

Use hashes or stable pseudonymous identifiers for tenant and subject dimensions when correlation is necessary. Do not use request IDs, user IDs, error messages, or raw URLs as metric labels because their high cardinality can overwhelm monitoring systems.

Tracing

Create an end-to-end trace with spans such as:

```

HTTP POST /mcp
├─ oauth.verify
│   └─ jwks.resolve_key
├─ mcp.parse
├─ mcp.dispatch tools/call
├─ fastmcp.middleware
├─ tool.get_order
│   ├── idempotency.lookup
│   ├── circuit.before_call
│   ├── http.orders_api attempt=1
│   ├── retry.backoff
│   ├── http.orders_api attempt=2
│   └─ response.validate
└─ mcp.serialize

```

Set span status to error for unexpected internal failures and dependency failures. For expected business outcomes such as “order not found,” prefer an outcome attribute without necessarily marking the entire infrastructure trace as faulty. OpenTelemetry traces are designed to represent the path of a request through distributed components and associate operations through trace and span identifiers. ³¹

Useful span attributes include:

```

rpc.system = "jsonrpc"
rpc.method = "tools/call"

```

```

mcp.tool.name = "get_order"
mcp.error.plane = "tool"
mcp.error.code = "ORDERS_DEPENDENCY_UNAVAILABLE"
mcp.tool.is_error = true
retry.attempt = 3
circuit.state = "closed"
auth.result = "success"
auth.scope_check = "success"
server.address = "mcp.example.com"
http.response.status_code = 200

```

Never attach bearer tokens or full tool arguments as span attributes.

Metrics and alerts

Metric	Type	Purpose
<code>mcp_requests_total{method,outcome}</code>	Counter	Overall request throughput and outcome
<code>mcp_jsonrpc_errors_total{code}</code>	Counter	Parse, request, method, parameter, and internal errors
<code>mcp_tool_calls_total{tool,outcome}</code>	Counter	Tool-level success and failure
<code>mcp_tool_duration_seconds{tool}</code>	Histogram	Tool latency distribution
<code>mcp_tool_queue_seconds{tool}</code>	Histogram	Concurrency saturation
<code>mcp_inflight_requests{tool}</code>	Gauge	Current concurrency
<code>mcp_retries_total{dependency,reason}</code>	Counter	Retry amplification
<code>mcp_retry_attempts{dependency}</code>	Histogram	Attempts per logical operation
<code>mcp_circuit_state{dependency}</code>	Gauge	Closed, half-open, or open
<code>mcp_circuit_transitions_total{dependency,to}</code>	Counter	Dependency instability
<code>mcp_rate_limit_rejections_total{layer}</code>	Counter	Gateway, tenant, tool, or dependency throttling
<code>oauth_validation_failures_total{reason}</code>	Counter	Expired, signature, issuer, audience, scope
<code>oauth_refresh_total{outcome}</code>	Counter	Refresh health
<code>oauth_refresh_reuse_detected_total</code>	Counter	Potential token compromise

Metric	Type	Purpose
<code>jwks_refresh_total{outcome}</code>	Counter	Key-discovery reliability
<code>idempotency_hits_total{result}</code>	Counter	New, replay, mismatch, in-progress
<code>process_event_loop_lag_seconds</code>	Gauge/histogram	Async runtime saturation

Alerts should be tied to the service's eventual SLA and normal baseline. With no specified SLA, no universal percentage threshold can be justified. At minimum, alert on sustained internal-error increases, open circuits, refresh-token replay, abrupt authentication-failure spikes, retry traffic approaching original traffic, JWKS refresh failure near key-cache expiry, high event-loop lag, queue-time growth, and exhaustion of worker or connection-pool capacity.

Track three rates separately:

```
protocol_error_rate =
    JSON-RPC protocol errors / parsed MCP requests

tool_failure_rate =
    tool results with isError=true / dispatched tool calls

internal_fault_rate =
    unexpected server exceptions / dispatched tool calls
```

Combining them into a single "error rate" obscures whether clients are malformed, dependencies are unstable, or application code is defective.

Testing strategy

Unit tests

Unit-test each classification boundary:

Unit under test	Cases
Input normalizer	Valid canonical input, controls, length, Unicode, traversal, malformed IDs
Error mapper	Every internal exception maps to the expected public code and retry flag
Retry classifier	Each HTTP status and exception class
Backoff	Bounds, jitter range, deadline exhaustion, <code>Retry-After</code>
Circuit breaker	Threshold, open rejection, half-open single probe, recovery

Unit under test	Cases
Idempotency	First request, exact replay, concurrent replay, argument mismatch
Token validator	Signature, issuer, audience, expiration, not-before, scopes
Redaction	Tokens and secrets absent from serialized logs
Output validator	Unexpected upstream fields, wrong type, oversized response

In-memory FastMCP integration tests

FastMCP supports an in-memory client connected directly to a server instance, eliminating subprocess and network overhead. This is well suited to testing tool-result semantics and exception masking. 32

```
import pytest

from fastmcp import Client

@pytest.mark.anyio
async def test_dependency_failure_is_model_actionable(monkeypatch):
    async def always_fails(**kwargs):
        raise DependencyUnavailable("injected outage")

    monkeypatch.setattr(
        "app.get_order_from_upstream",
        always_fails,
    )

    client = Client(mcp)

    async with client:
        result = await client.call_tool(
            "get_order",
            {
                "order_id": "ORD-123",
                "idempotency_key": str(uuid.uuid4()),
            },
            raise_on_error=False,
        )

    assert result.is_error is True
    assert result.structured_content["error"]["code"] == (
        "ORDERS_DEPENDENCY_UNAVAILABLE"
    )
    assert result.structured_content["error"]["retryable"] is True
```

```

@pytest.mark.anyio
async def test_safe_tool_error_is_propagated():
    client = Client(mcp)

    async with client:
        result = await client.call_tool(
            "get_order",
            {
                "order_id": "../etc/passwd",
                "idempotency_key": str(uuid.uuid4()),
            },
            raise_on_error=False,
        )

    assert result.is_error is True
    rendered = " ".join(
        block.text
        for block in result.content
        if getattr(block, "type", None) == "text"
    )
    assert "Invalid order_id" in rendered
    assert "/etc/passwd" not in rendered


@pytest.mark.anyio
async def test_unexpected_details_are_masked(monkeypatch):
    async def programming_bug(**kwargs):
        raise RuntimeError(
            "postgresql://admin:secret@db.internal/orders"
        )

    monkeypatch.setattr(
        "app.get_order_from_upstream",
        programming_bug,
    )

    client = Client(mcp)

    async with client:
        result = await client.call_tool(
            "get_order",
            {
                "order_id": "ORD-123",
                "idempotency_key": str(uuid.uuid4()),
            },
            raise_on_error=False,
        )

```

```
rendered = repr(result.content)
assert "postgresql://" not in rendered
assert "secret" not in rendered
```

FastMCP's documented client behavior supports `raise_on_error=False` and `result.is_error`, which makes assertions on tool failures straightforward. ³³

HTTP and protocol integration tests

Test the deployed HTTP transport with raw requests, not only the high-level client:

Scenario	Assertion
Truncated JSON	HTTP 400 and JSON-RPC <code>-32700</code>
Valid JSON, invalid JSON-RPC	<code>-32600</code>
Unknown method	<code>-32601</code>
Invalid tool arguments	<code>-32602</code> or documented error contract
Notification without <code>id</code>	No JSON-RPC response
Duplicate IDs on concurrent connections	Responses remain correctly correlated
Tool <code>isError</code>	HTTP request succeeds; body contains <code>result.isError=true</code>
Missing token	401 and parseable <code>WWW-Authenticate</code>
Expired token	401 <code>invalid_token</code>
Missing scope	403 <code>insufficient_scope</code>
Invalid refresh token	Token endpoint returns <code>invalid_grant</code>
Revocation of invalid token	Revocation endpoint still returns 200
Oversized body	413 before tool dispatch
Unsupported media type	415
Reverse-proxy base URL	Metadata and redirect URLs use the public origin
CORS/Origin/Host	Only intended clients and hosts are accepted

JSON-RPC notifications must not receive a response, even when processing encounters an error. ⁶

Fault injection

Inject failures at multiple layers:

Dependency faults:

- connection refused
- DNS failure
- TLS failure
- connect timeout
- slow headers
- partial body
- invalid JSON
- 429 with and without Retry-After
- 500, 502, 503, 504
- success after one or two failures

OAuth faults:

- JWKS timeout
- JWKS containing supported and unsupported key types
- key rotation between requests
- unknown kid
- wrong issuer or audience
- expired and not-yet-valid token
- missing scope
- revoked access token
- concurrent refresh requests
- refresh-token reuse

Runtime faults:

- event-loop delay
- connection-pool exhaustion
- worker termination during a write
- cancellation during retry sleep
- cancellation after upstream commit
- multiple identical idempotent requests
- dependency recovery while circuit is open

The mixed-JWKS case is particularly important for 3.4.5 because skipping unsupported JWKS keys is the release's leading fix. [5](#)

Property and fuzz testing

Generate:

- arbitrary JSON values rather than only objects
- extremely nested arrays and objects
- very large strings and numeric values
- duplicate or unusual Unicode keys
- invalid UTF-8 at the HTTP boundary
- path and URL encodings

unexpected JSON-RPC ids
schema-valid but semantically impossible values

Assertions should include bounded processing time, bounded memory use, no crashes, no secret leakage, and a response belonging to the documented error taxonomy.

Scalability and concurrency

FastMCP HTTP tools can be asynchronous, but async alone does not provide overload protection. Add:

Control	Purpose
Per-process in-flight limit	Prevent event-loop and memory exhaustion
Per-tool semaphore	Isolate expensive tools from inexpensive ones
Per-tenant quota	Prevent one tenant from monopolizing capacity
Dependency-specific connection pool	Bound sockets and upstream concurrency
Queue deadline	Reject calls that waited too long to remain useful
Request deadline propagation	Prevent downstream work after the caller has timed out
Bulkhead isolation	Contain failure of one dependency or tool class
Backpressure	Fail fast rather than accepting unbounded work
Graceful draining	Stop admitting new calls and finish bounded in-flight work
Shared state	Coordinate distributed idempotency and rate limits across replicas

Avoid blocking filesystem, database, cryptographic, or HTTP work on the event loop. Move truly blocking operations to a bounded thread or process pool. A thread pool without a queue limit merely moves the overload point.

Circuit-breaker and idempotency state must match the deployment topology:

Deployment	State design
Single process	In-memory breaker is possible; durable idempotency still preferred for writes
Multiple workers on one host	Breaker may be per worker; idempotency and quotas need shared storage
Multiple replicas	Distributed idempotency, quotas, and possibly aggregated circuit health

Deployment	State design
Multi-region	Region-local circuit and rate state; globally durable operation identity where cross-region retries occur

Do not make every request synchronously depend on a global circuit-breaker database. Local breakers can protect individual workers, while shared metrics and a gateway provide broader coordination.

Rate limiting

Apply limits at more than one layer:

Edge/gateway:

body size, connection rate, total requests, IP abuse

Authentication:

token endpoint, DCR, authorization attempts, JWKS or introspection calls

MCP:

requests per client/tenant, tools/list frequency, concurrent sessions

Tool:

per-tool cost, expensive query fan-out, write rate

Dependency:

upstream quota and connection-pool budget

A gateway-level rejection before an MCP request is accepted should use HTTP 429 with `Retry-After`. A valid tool invocation that encounters an upstream quota is usually better returned as `isError: true`, including a safe retry interval, because the MCP client can understand that the tool—not the JSON-RPC transport—was rate-limited.

Retries consume quota. Monitor:

```
retry amplification =
  total dependency attempts / original logical dependency operations
```

A value rising toward two means the server is approximately doubling load on an already impaired dependency.

Deployment concerns

Production deployment should verify:

Concern	Required decision
Public URL	OAuth metadata, audience, resource, and callback URLs use the externally visible HTTPS origin
Reverse proxy	Trusted forwarding headers are explicit; arbitrary clients cannot spoof them
Host/Origin	Explicit allow-lists where deployment topology supports them
Health checks	Liveness checks process health; readiness reflects ability to accept useful traffic
Shutdown	Stop admission, allow bounded draining, close HTTP clients and exporters
Workers	Capacity derived from workload and blocking behavior, not CPU count alone
Sessions	Shared storage or appropriate routing if session state is used
Secrets	Managed secret store, rotation, least-privileged runtime identity
Logging	Structured output, redaction, retention, access controls
TLS	Termination and internal transport threat model documented
Dependency pools	Pool sizes coordinated with worker and replica counts
Rollout	Canary deployment and rapid rollback
Version	Exact FastMCP pin and dependency lockfile

FastMCP 3.4.4's restoration of compatibility after stricter Host/Origin defaults is a reason to test the real production proxy path. Security checks that work locally can behave differently when the application is mounted under a prefix, behind TLS termination, or deployed through a serverless adapter. ⁵

Information needed to finalize the design

The report assumes **no specific constraint** for all unspecified items. The following information is needed to convert the reference architecture into an exact production design:

Information required	Why it changes the design	Current assumption
Primary server language	Determines whether Python FastMCP is the runtime or Node is a gateway/client	Python FastMCP 3.4.5 is authoritative; Node is protocol-equivalent
Python and Node runtime versions	Affects async, dependency, security, and deployment support	No specific constraint
Web framework and mounting model	Determines ASGI middleware, proxy headers, lifecycle, and status mapping	Standalone FastMCP HTTP
MCP transport	OAuth applies to HTTP/SSE, not the same way to stdio	Streamable HTTP for remote service

Information required	Why it changes the design	Current assumption
MCP protocol version and client population	Determines compatibility testing and session behavior	No specific constraint
OAuth topology	<code>JWTVerifier</code> , <code>RemoteAuthProvider</code> , <code>OAuthProxy</code> , or custom provider	External issuer plus JWT verification
Identity provider	Determines discovery, DCR, PKCE, token format, JWKS, and refresh behavior	Standards-compatible OAuth/OIDC provider
Access-token format	JWT versus opaque token changes local verification versus introspection	JWT
Required issuer, audience, resource, and scopes	Needed for exact token validation	No specific values
Refresh-token policy	Determines rotation, storage, replay handling, and session duration	Rotation or sender-constraining preferred
Revocation mechanism	Determines incident response and logout semantics	RFC 7009-compatible endpoint
Tool catalog and side effects	Determines which tools can be retried and need idempotency	Unknown
Upstream dependencies	Determines retryable statuses, timeout budgets, and circuits	Generic HTTP APIs
Expected request rate	Determines workers, pools, quotas, and metric scale	No specific constraint
Expected error rates	Needed for alert baselines and circuit thresholds	No specific constraint
Concurrency target	Determines semaphores, queue limits, and resource pools	No specific constraint
Upstream rate limits	Determines token buckets, <code>Retry-After</code> , and retry budgets	No specific constraint
Retry policy	Maximum attempts, deadlines, jitter, and budgets	Three attempts within an eight-second dependency deadline in examples
Idempotency retention	Determines storage size, TTL, replay, and late-arrival behavior	No specific constraint
SLA and SLOs	Determines timeout, availability, latency, and alert thresholds	No specific constraint

Information required	Why it changes the design	Current assumption
Recovery time and recovery point objectives	Determines durable state and regional failover	No specific constraint
Data sensitivity	Determines logging, encryption, retention, and audit controls	Treat tool data and tokens as sensitive
Tenant model	Determines authorization, quotas, circuit isolation, and log fields	Multi-tenant-safe design recommended
Deployment platform	Determines process model, health checks, secret injection, and scaling	Containerized HTTP service
Reverse proxy or API gateway	Determines trusted headers, auth offload, body limits, and 429 behavior	Present but unspecified
Observability platform	Determines OpenTelemetry exporter, log schema, and metric naming	OpenTelemetry-compatible
Compliance requirements	Determines audit retention, access reviews, key management, and incident procedure	No specific constraint
Client error contract	Determines stable machine codes and backward-compatibility commitments	Proposed <code>ok/error</code> envelope
User-facing disclosure policy	Determines which business details are safe to expose	Only actionable, non-sensitive details
Human confirmation requirements	Needed for destructive or high-impact tools	Confirmation required for sensitive operations

The central architectural recommendation is to maintain three explicit boundaries: **HTTP/OAuth failures before MCP dispatch, JSON-RPC errors for malformed or undispachable protocol operations, and `ToolResult(is_error=True)` or `ToolError` for valid calls whose execution failed**. FastMCP 3.4.5 supplies the necessary server primitives; reliability and security depend on applying them consistently with bounded retries, durable idempotency, dependency isolation, safe token handling, and observable error classification.

1 5 Changelog - FastMCP

<https://gofastmcp.com/changelog>

2 Tools - Model Context Protocol

<https://modelcontextprotocol.io/specification/2025-06-18/server/tools>

3 8 fastmcp/fastmcp_slim/fastmcp/exceptions.py at v3.4.5 · PrefectHQ/fastmcp · GitHub

https://github.com/PrefectHQ/fastmcp/blob/v3.4.5/fastmcp_slim/fastmcp/exceptions.py

- 4 punkpeye/fastmcp: A TypeScript framework for building ...
https://github.com/punkpeye/fastmcp?utm_source=chatgpt.com
- 6 21 JSON-RPC 2.0 Specification
<https://www.jsonrpc.org/specification>
- 7 fastmcp/fastmcp_slim/fastmcp/tools/base.py at v3.4.5 · PrefectHQ/fastmcp · GitHub
https://github.com/PrefectHQ/fastmcp/blob/v3.4.5/fastmcp_slim/fastmcp/tools/base.py
- 9 10 19 Tools - FastMCP
<https://gofastmcp.com/servers/tools>
- 11 fastmcp/fastmcp_slim/fastmcp/tools/function_tool.py at v3.4.5 · PrefectHQ/fastmcp · GitHub
https://github.com/PrefectHQ/fastmcp/blob/v3.4.5/fastmcp_slim/fastmcp/tools/function_tool.py
- 12 MCP Middleware - FastMCP
<https://gofastmcp.com/v2/servers/middleware>
- 13 33 Calling Tools - FastMCP
<https://gofastmcp.com/clients/tools>
- 14 Authorization Server Discovery
https://modelcontextprotocol.io/specification/2026-07-28/basic/authorization/authorization-server-discovery?utm_source=chatgpt.com
- 15 Timeouts, retries, and backoff with jitter
https://aws.amazon.com/builders-library/timeouts-retries-and-backoff-with-jitter/?utm_source=chatgpt.com
- 16 references-details-empty
<https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/>
- 17 Input Validation - OWASP Cheat Sheet Series
https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html
- 18 Token Verification - FastMCP
<https://gofastmcp.com/servers/auth/token-verification>
- 20 GitHub - modelcontextprotocol/typescript-sdk: The official TypeScript SDK for Model Context Protocol servers and clients · GitHub
<https://github.com/modelcontextprotocol/typescript-sdk>
- 22 www.rfc-editor.org
<https://www.rfc-editor.org/rfc/rfc6750.html>
- 23 RFC 9728: OAuth 2.0 Protected Resource Metadata
<https://www.rfc-editor.org/rfc/rfc9728.html>
- 24 www.rfc-editor.org
<https://www.rfc-editor.org/rfc/rfc6749.html>
- 25 Authentication - FastMCP
<https://gofastmcp.com/servers/auth/authentication>
- 26 OAuth Proxy - FastMCP
<https://gofastmcp.com/servers/auth/oauth-proxy>

27 RFC 9700: Best Current Practice for OAuth 2.0 Security

<https://www.rfc-editor.org/rfc/rfc9700.html>

28 www.rfc-editor.org

<https://www.rfc-editor.org/rfc/rfc7009.html>

29 Client Logging - FastMCP

<https://gofastmcp.com/v2/servers/logging>

30 Logging - OWASP Cheat Sheet Series

https://cheatsheetseries.owasp.org/cheatsheets/Logging_Cheat_Sheet.html

31 Traces | OpenTelemetry

<https://opentelemetry.io/docs/concepts/signals/traces/>

32 Client Transports - FastMCP

<https://gofastmcp.com/clients/transport>